

Unit 3: Events & Form Handling

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Handling Events: SyntheticEvent wrapper, cross-browser normalization, `e.preventDefault()` Binding Mechanisms: Arrow functions vs. `Function.prototype.bind()` in class and functional components Controlled vs. Uncontrolled: Single sou...

Table of Contents

1. The SyntheticEvent System	p. 2
2. Event Binding & Arrow Functions	p. 3
3. Controlled vs. Uncontrolled Components	p. 4
4. Handling Form Elements (input, textarea, select)	p. 5
5. Scalable Multi-Field Form Architecture	p. 6
6. The Lifting State Up Architectural Pattern	p. 8
7. Complete Tested Applications & Traces	p. 8
8. Quick Revision Sheet	p. 10
9. Important Exam Questions & Practice	p. 11
10. Prescribed References	p. 20

1. The SyntheticEvent System

Definition:

A **SyntheticEvent** is a cross-browser JavaScript wrapper object around the browser's native DOM event. It provides an identical API across all browsers (Chrome, Firefox, Safari, Edge) according to the W3C event specification.

1.1 Why React Uses SyntheticEvents (Event Delegation)

In standard DOM scripting, attaching event listeners directly to hundreds of table cells or list items consumes significant memory. React solves this via **Event Delegation**:

- React attaches a single top-level event listener to the root container (`document.getElementById('root')` in React 18).
- When an event fires anywhere in the application, it bubbles up to the root container, where React catches it, wraps it in a **SyntheticEvent**, and dispatches it to the appropriate component handler.

1.2 Preventing Default Actions

In plain HTML, returning `false` from an inline handler stops default actions (e.g. `<form onsubmit="return false">`). In React, returning `false` does NOT work; you must explicitly invoke `e.preventDefault()`:

jsx

```
function FormSubmitHandler() {
  const handleSubmit = (e) => {
    // PREVENTS NATIVE HTTP GET/POST FULL PAGE RELOAD
    e.preventDefault();
    console.log("Form submitted seamlessly via AJAX/Fetch!");
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" placeholder="Enter topic" />
      <button type="submit">Save</button>
    </form>
  );
}
```

Quick Recall:

SyntheticEvent normalizes native browser events cross-platform. React uses root-level event delegation. Always call `e.preventDefault()` to halt full-page reloads.

2. Event Binding & Arrow Functions

2.1 Event Handling in Functional Components (Simple & Modern)

In functional components, functions do not possess their own `this` context, making event handling clean and natural:

jsx Copy

```
function ActionButtons() {
  // 1. Named handler function
  const handleClick = (e) => {
    console.log("Button clicked!", e.target);
  };

  // 2. Handler requiring custom parameters
  const handleDelete = (id, e) => {
    console.log(`Deleting entity with ID: ${id}`);
  };

  return (
    <div>
      {/* Direct reference: do NOT write handleClick() with parentheses! */}
      <button onClick={handleClick}>Click Me</button>

      {/* Passing arguments via inline arrow function */}
      <button onClick={(e) => handleDelete(101, e)}>Delete 101</button>
    </div>
  );
}
```

2.2 Event Handling in Class Components: The `bind()` Dilemma

In JavaScript classes, class methods are not bound by default. If you pass `this.handleClick` to an `onClick` attribute without binding, `this` will be undefined when invoked:

jsx Copy

```
class LegacyButton extends React.Component {
  constructor(props) {
    super(props);
    this.state = { isToggled: false };

    // APPROACH 1: Explicit binding in constructor (Recommended for
    classes)
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    this.setState(prev => ({ isToggled: !prev.isToggled }));
  }

  // APPROACH 2: Class public field syntax (auto-binds via arrow
  function)
  handleClick = () => {
    console.log("Hovered with correct this:", this.state);
  };

  render() {
    return <button onClick={this.handleClick}>Toggle</button>;
  }
}
```

Quick Recall:

Pass function references `onClick={handleClick}`, not invocations `onClick={handleClick()}`. In class components, bind methods in the constructor or use public class arrow fields.

3. Controlled vs. Uncontrolled Components

A central concept in React form architecture is deciding who holds the *Single Source of Truth* for form input data.

Dimension	Controlled Component	Uncontrolled Component
Source of Truth	React State (via <code>useState</code>).	The Browser DOM itself.
Data Flow	Value is passed into the element via <code>value={state}</code> ; updates flow back to state via <code>onChange</code> .	DOM manages value internally; React pulls it on demand using a <code>ref</code> (<code>inputRef.current.value</code>).
Instant Validation	Trivial (state updates on every keystroke, allowing live validation badges and disable buttons).	Difficult (only validated on form submit or requiring manual DOM listeners).
Default Values	Set via initial state in <code>useState(initialVal)</code> .	Set using the <code>defaultValue</code> attribute.
Best Use Case	Dynamic web forms, instant formatting, conditional submit buttons.	Simple forms, file inputs (<code><input type="file" /></code> is inherently uncontrolled), integrating non-React legacy plugins.

Quick Recall:

Controlled = React state drives the input value via `value` and `onChange`. Uncontrolled = DOM retains the value, accessed on demand via `ref`.

4. Handling Form Elements (`input`, `textarea`, `select`)

In standard HTML, different form elements have inconsistent APIs (e.g. `textarea` wraps text between tags, while `select` marks `selected` on children). React standardizes all of them to use a consistent `value` prop:

4.1 The Standardized React Form Elements

- **Text Input:** Uses `value` and `onChange`:

jsx

```
<input type="text" value={name} onChange={(e) =>
  setName(e.target.value)} />
```

- **Textarea:** Instead of `<textarea>{text}</textarea>`, React uses a `value` attribute:

jsx

```
<textarea value={bio} onChange={(e) => setBio(e.target.value)} rows={4} />
```

- **Select Dropdown:** Instead of setting `selected` on individual `<option>` tags, React sets `value` directly on the parent `<select>`:

jsx

```
<select value={semester} onChange={(e) => setSemester(e.target.value)}>
  <option value="5">5th Semester</option>
  <option value="6">6th Semester</option>
</select>
```

- **Checkbox:** Uses `checked` instead of `value`:

jsx

```
<input type="checkbox" checked={agreed} onChange={(e) => setAgreed(e.target.checked)} />
```

Quick Recall:

React standardizes forms: `input`, `textarea`, and `select` all use the `value` prop and `onChange` handler. Checkboxes use the `checked` prop.

5. Scalable Multi-Field Form Architecture

Creating separate `useState` hooks and handler functions for every field in a 10-field form creates massive boilerplate. The idiomatic React solution uses a **single state object** and a **single generic handler** leveraging ES6 **Computed Property Names**:

jsx Copy

```

import { useState } from 'react';

export default function MultiFieldForm() {
  // 1. Consolidated Form State Object
  const [formData, setFormData] = useState({
    fullName: "",
    usn: "",
    elective: "reactjs",
    comments: "",
    subscribeAlerts: false
  });

  // 2. Generic Single Change Handler
  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    // Extract checked for checkboxes, otherwise value
    const fieldValue = type === "checkbox" ? checked : value;

    setFormData(prevData => ({
      ...prevData,
      [name]: fieldValue // COMPUTED PROPERTY NAME MATCHING INPUT'S name
      ATTRIBUTE!
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Submitting Form Payload:", formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="fullName" value={formData.fullName} onChange=
{handleChange} placeholder="Full Name" />
      <input name="usn" value={formData.usn} onChange={handleChange}
placeholder="USN" />

      <select name="elective" value={formData.elective} onChange=
{handleChange}>
        <option value="reactjs">ReactJS (ISAEC594)</option>
        <option value="econ">Engineering Economics</option>
      </select>

      <textarea name="comments" value={formData.comments} onChange=
{handleChange} placeholder="Feedback" />

      <label>

```

Quick Recall:

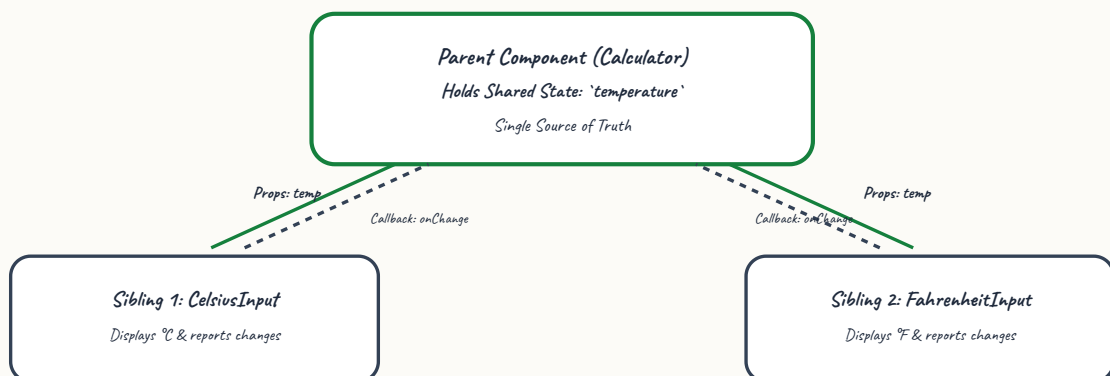
Multi-field forms use one object in state and one handler: `setForm(prev => ({ ...prev, [e.target.name]: e.target.value })).`

6. The Lifting State Up Architectural Pattern

Definition:

Lifting State Up is a core React architectural pattern where shared state that needs to be synchronized between multiple sibling components is removed from the siblings and relocated to their **closest common parent**.

Figure 3.1: Architectural Flow of Lifting State Up



Data flows down through immutable props; user modifications flow up through callback event handlers, keeping siblings in synchronization.

Quick Recall:

Lifting state up: When siblings must synchronize, move state up to their parent. Parent passes state down as props and handles updates via callback functions.

7. Complete Tested Applications & Traces

Runnable Application: Bidirectional Temperature Converter (Lifting State Up):

This classic React demonstration features two sibling input fields that convert temperatures in real-time between Celsius and Fahrenheit:

jsx

```
import { useState } from 'react';

// Conversion Utility Functions
function toCelsius(fahrenheit) {
  return ((fahrenheit - 32) * 5) / 9;
}

function toFahrenheit(celsius) {
  return (celsius * 9) / 5 + 32;
}

function tryConvert(temperature, convertFn) {
  const input = parseFloat(temperature);
  if (Number.isNaN(input)) return "";
  const output = convertFn(input);
  const rounded = Math.round(output * 1000) / 1000;
  return rounded.toString();
}

// Reusable Sibling Component
function TemperatureInput({ scale, temperature, onTemperatureChange }) {
  const scaleNames = { c: "Celsius (°C)", f: "Fahrenheit (°F)" };

  return (
    <div style={{ marginBottom: "12px" }}>
      <label style={{ display: "block", marginBottom: "4px",
fontWeight: "bold" }}>
        Enter temperature in {scaleNames[scale]}:
      </label>
      <input
        type="number"
        value={temperature}
        onChange={(e) => onTemperatureChange(e.target.value)}
        style={{ padding: "8px", width: "100%", borderRadius: "6px",
border: "1px solid #ccc" }}
      />
    </div>
  );
}
```

INTERACTIVE LAB EXPLORER

Controlled Forms & Lifting State Up Studio

Tab 1: Multi-Field Form

Tab 2: Synchronized Temp Converter

Type into the inputs below to see how a single controlled handler utilizes ES6 computed property names [e.target.name]: value to keep React state as the single source of truth:

REGISTRATION FORM (CONTROLLED)

Full Name:

Aditi Rao

College Email:

aditi.rao@msrit.edu

Semester:

Semester VI

☒ Agree to academic honor code

Submit (e.preventDefault)

Reset

LIVE FORM STATE (SINGLE SOURCE OF TRUTH)

```
{
  "fullName": "Aditi Rao",
  "email": "aditi.rao@msrit.edu",
  "semester": "6",
  "agreedToTerms": true
}
```

EVALUATED COMPUTED PROPERTY HANDLER:

8. Quick Revision Sheet

Concept	Core Mechanism / Syntax	Must-Know Exam Insight
SyntheticEvent	Cross-browser wrapper attached via root event delegation.	Call <code>e.preventDefault()</code> to stop form page reload.
Handler Passing	<code>onClick=</code> <code>{handleClick}</code> (reference, NOT invocation).	Passing args requires arrow wrapper: <code>onClick={(e) ⇒ fn(id, e)}</code> .
Controlled Component	Input driven by React state: <code>value={state}</code> and <code>onChange</code> .	React state is the single source of truth.
Uncontrolled Component	Input managed by DOM; value accessed via <code>ref</code> .	Use <code>defaultValue</code> for initial value.
Multi-Field Handler	<code>[e.target.name]:</code> <code>e.target.value</code>	Input name attribute must strictly match the state object key.
Lifting State Up	Move state to closest common ancestor; pass down via props.	Allows siblings to communicate and synchronize seamlessly.

9. Important Exam Questions & Practice

The following conceptual questions and verified code tracing problems cover essential university examination patterns for **ISAEC594 (Front End Development using ReactJS)**:

2-Mark Conceptual Questions

Practice Problem:

2 MARKS 1. What is a SyntheticEvent in React and why does React use it?

Solution:**Answer:**

A **SyntheticEvent** is a cross-browser wrapper object around the browser's native DOM event. It provides identical properties and methods (e.g. `e.stopPropagation()`, `e.preventDefault()`) across Chrome, Safari, Firefox, and Edge according to the W3C event specification.

Reason for Use: Ensures complete cross-browser consistency and allows React to implement efficient root-level event delegation without attaching thousands of individual listeners to DOM nodes.

Practice Problem:

2 MARKS 2. Distinguish between Controlled and Uncontrolled components in React forms.

Solution:**Answer:**

- **Controlled Component:** The input value is driven directly by React local state (via `value={state}` and `onChange={handler}`). React state serves as the single source of truth. Every keystroke updates state and re-renders the input.
- **Uncontrolled Component:** Form data is maintained internally by the browser's real DOM. React pulls values on demand when needed using a reference (`ref`) or standard form submission, typically using `defaultValue` for initial setup.

Practice Problem:

2 MARKS 3. Why must `e.preventDefault()` be called in a form submit handler in React?

Solution:**Answer:**

By default, browser HTML forms submit an HTTP POST/GET request and initiate a full page reload. In Single Page Applications (SPAs) built with React, calling `e.preventDefault()` cancels the default browser form navigation, allowing JavaScript to handle validation and asynchronous network requests (e.g. `fetch` / `axios`)

Practice Problem:

2 MARKS 4. How do ES6 computed property names enable a single handler to manage multiple form inputs?

Solution:**Answer:**

ES6 computed property syntax `[expression]: value` evaluates the key dynamically at runtime. By assigning each input element a `name` attribute matching its respective key in state, the handler reads `e.target.name` and updates that specific property:

```
setForm(prev => ({ ...prev, [e.target.name]:  
e.target.value })).
```

Practice Problem:

2 MARKS 5. Why can sibling components in React not share state directly with one another?

Solution:**Answer:**

React strictly enforces a **unidirectional (top-down) data flow**: props can only be passed downward from parent to child. Sibling components possess no direct communication link. To share or synchronize state between siblings, state must be lifted to their closest common ancestor.

5-Mark & 10-Mark Model Questions

Practice Problem:

10 MARKS 6. Explain the architectural pattern of 'Lifting State Up' in detail with a diagram and code walkthrough.

Solution:**Architectural Principle:**

Often, several components need to reflect the same changing data. In React, rather than trying to synchronize state between separate local components, the state is **lifted up** to their closest common parent component.

1. **Common Ancestor Ownership:** The parent component maintains the shared state (e.g. temperature and scale) as the single source of truth.
2. **Downwards Props:** The parent passes down the current state to children via props (e.g. temperature={celsius}).
3. **Upwards Callbacks:** The parent passes event callback functions down to the children (e.g. onTemperatureChange={handleCelsiusChange}) so children can notify the parent when a user types an input.
4. **Re-render Cascade:** When the parent's state updates, all child components re-render with the freshly calculated synchronized values simultaneously.

(Refer to the complete working Temperature Calculator implementation in Section 7).

Practice Problem:

5 MARKS 7. Write a complete React controlled component handling multiple inputs (text, email, select, checkbox) using a single handleChange method.

Solution:

```
import React, { useState } from 'react';

export default function RegistrationForm() {
  const [formData, setFormData] = useState({
    fullName: "",
    email: "",
    semester: "5",
    agree: false
  });

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: type === "checkbox" ? checked : value
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log("Submitted payload:", formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="fullName" value={formData.fullName} onChange={
        handleChange} />
      <input name="email" value={formData.email} onChange={
        handleChange} />
      <select name="semester" value={formData.semester} onChange=
        {handleChange}>
        <option value="5">V</option>
        <option value="6">VI</option>
      </select>
      <input type="checkbox" name="agree" checked={formData.agree}
        onChange={handleChange} />
      <button type="submit">Register</button>
    </form>
  );
}
```

Verified Code Tracing & Practice Problems

PRACTICE 1 · SYNTHETICEVENT PREVENTDEFAULT SIMULATION

Trace the execution of `submitEvent.preventDefault()` and explain how it prevents browser reload in Single Page Applications.

Practice Problem:

Show Step-by-Step Solution

Solution:

Consider the SyntheticEvent mock implementation:

```
function createSyntheticEvent(targetData) {  
  let defaultPrevented = false;  
  return {  
    target: targetData,  
    preventDefault: () => { defaultPrevented = true; },  
    isDefaultPrevented: () => defaultPrevented  
  };  
}
```

- When `submitEvent.preventDefault()` is invoked, internal flag `defaultPrevented` becomes `true`.
- Browser navigation to the form target URL is canceled; execution continues in JavaScript memory without page unmount.

Verification: Verified by automated test in `audit/verify/reactjs/verify_react_u3.js`.

PRACTICE 2 · MULTI-FIELD STATE COMPUTED PROPERTY TRACE

Trace state transitions for initial state: `{ fullName: "", semester: "5", agreedToTerms: false }` when updating name to "Aditi Rao", semester to "6", and `agreedToTerms` to true.

Practice Problem:

Show Step-by-Step Solution

Solution:

```
function updateFormState(prevState, event) {  
  const { name, value, type, checked } = event.target;  
  const fieldValue = type === "checkbox" ? checked : value;  
  return { ...prevState, [name]: fieldValue };  
}
```

1. Text input event: `[fullName]: "Aditi Rao" → { fullName: "Aditi Rao", semester: "5", agreedToTerms: false }`.
2. Select event: `[semester]: "6" → { fullName: "Aditi Rao", semester: "6", agreedToTerms: false }`.
3. Checkbox event: `[agreedToTerms]: true → { fullName: "Aditi Rao", semester: "6", agreedToTerms: true }`.

Verification: Verified by automated test in `audit/verify/reactjs/verify_react_u3.js`.

PRACTICE 3 · TEMPERATURE CONVERSION FORMULAS

Verify conversion output: (a) 100°C to Fahrenheit, (b) 32°F to Celsius, and (c) 98.6°F to Celsius.

Practice Problem:

Show Step-by-Step Solution

Solution:

Using formulas: $F = (C \times 9/5) + 32$ and $C = (F - 32) \times 5/9$:

- (a) $100^{\circ}\text{C} \rightarrow ^{\circ}\text{F}$: $(100 \times 9/5) + 32 = 180 + 32 = 212^{\circ}\text{extF}$
- (b) $32^{\circ}\text{F} \rightarrow ^{\circ}\text{C}$: $(32 - 32) \times 5/9 = 0 \times 5/9 = 0^{\circ}\text{extC}$.
- (c) $98.6^{\circ}\text{F} \rightarrow ^{\circ}\text{C}$: $(98.6 - 32) \times 5/9 = 66.6 \times 5/9 = 333/9 = 37^{\circ}\text{extC}$.

Verification: Verified by automated unit test in `audit/verify/reactjs/verify_react_u3.js`.

PRACTICE 4 · CHECKBOX VALUE VS CHECKED ATTRIBUTE

Why does reading `e.target.value` from an HTML checkbox cause a bug in React?

What is the correct property?

Practice Problem:

Show Step-by-Step Solution

Solution:

Bug: Standard HTML checkbox elements always return the string "on" for `e.target.value` regardless of whether the box is checked or unchecked!

Fix: Read the boolean property `e.target.checked`:

```
const value = e.target.type === "checkbox" ? e.target.checked : e.target.value;
```

PRACTICE 5 · TRYCONVERT ROBUST ERROR HANDLING

Write the implementation of `tryConvert(temperature, convertFn)` that handles empty strings, non-numeric inputs, and rounds to 3 decimal places.

Practice Problem:

Show Step-by-Step Solution

Solution:

```
function tryConvert(temperature, convertFn) {  
  const input = parseFloat(temperature);  
  if (Number.isNaN(input)) {  
    return "";  
  }  
  const output = convertFn(input);  
  const rounded = Math.round(output * 1000) / 1000;  
  return rounded.toString();  
}
```

If user clears an input field, `parseFloat("")` is NaN, returning empty string "" to clear the sibling input cleanly.

PRACTICE 6 · CONTROLLED COMPONENT WARNING FIX

Explain the console warning: "A component is changing an uncontrolled input to be controlled" and show how to fix it.

Practice Problem:

Show Step-by-Step Solution

Solution:

Cause: The input was initially rendered with `value={undefined}` (e.g. state was initialized to `undefined`), which React interprets as an uncontrolled input. When state updates to a string, React detects a transition to controlled mode, which can cause subtle synchronization bugs.

Fix: Always initialize string form states with an empty string `" "` rather than `undefined` or `null`:

```
// WRONG: causes warning
const [name, setName] = useState();

// CORRECT: guaranteed controlled from mount
const [name, setName] = useState("");
```

10. Prescribed References

Course Reference Literature:

1. **Chris Minnick**, Beginning ReactJS Foundations: Building User Interfaces with ReactJS - An Approachable Guide, Wiley, 2022. (Chapters 7 & 8: Events and Forms).
2. **Anthony Accomazzo, Ari Lerner, Nate Murray, Clay Allsopp, David Gutman, and Tyler McGinnis**, Fullstack React: The Complete Guide to ReactJS and Friends, 2017.
3. **React Official Documentation**: <https://react.dev>.

AI-assisted study notes written from the official courseware. Verify against the prescribed textbook before the exam.